

EGCO 425 – Chapter 9

Ensemble Methods

References

- [1] Tan et al., Introduction to Data Mining (Chapter 5)
- [2] Han & Kamber, Data Mining: Concepts and Techniques (Chapter 8)
- [3] Kotu & Deshpande, Data Science: Concepts and Practice (Chapter 4)
- [4] RapidMiner Doc, <https://docs.rapidminer.com/latest/studio/operators/>

Chapter Outline

- ⊕ Individual vs. ensemble classifiers
- ⊕ Ensemble methods
 - ▶ Voting, stacking
 - ▶ Bagging, random forest
 - ▶ Boosting (AdaBoost), gradient boosting tree
 - ▶ Multi-class to binary classification

Wisdom of the crowd

- ⊕ Suppose that, for a 2-class problem
 - ▶ We have 25 base classifiers, each of them has error rate $e = 0.35$
 - ▶ In ensemble model, prediction is based on majority vote (at least 13 out of 25). Error occurs if 13, 14, ..., 25 classifiers make wrong choice

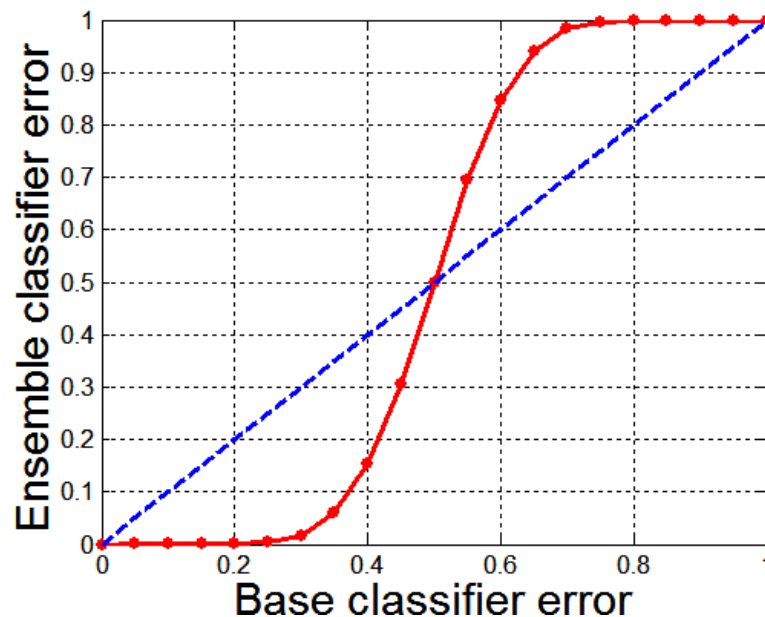
- ⊕ If base classifier is independent of each other

$$e_{ensemble} = \sum_{i=13}^{25} \binom{25}{i} e^i (1-e)^{25-i}$$

$$= 0.06$$

Weak classifier = base classifier

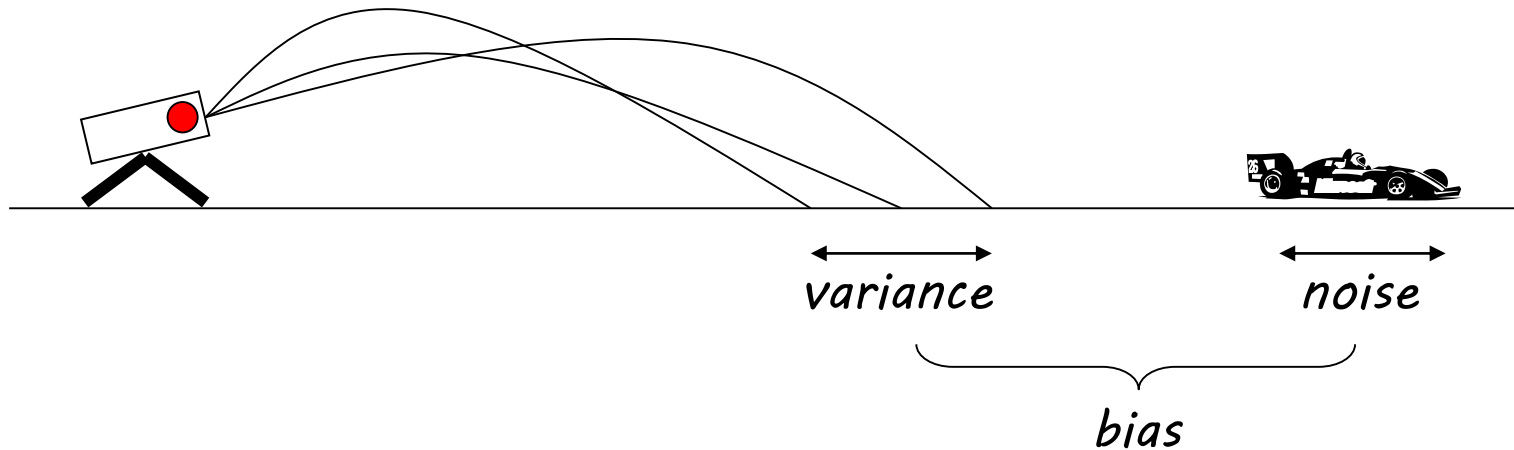
Strong classifier = ensemble classifier



When is ensemble classifier better than individual one ?

- ⊕ Each base classifier does better than random guessing (error < 0.5)
- ⊕ Base classifier is independent of each other
 - ▶ Difficult to prove in real practice. But there are reports that slightly correlated classifiers still make a good ensemble classifier
- ⊕ Base classifier is sensitive to minor changes in training data
 - ▶ Any little change may result in completely different model
 - ▶ Reasons : data set is too small, contain noises, much variation in data (inherent characteristics of the data itself)
- ⊕ Too many attributes for just 1 classifier

Source of classifier's variation



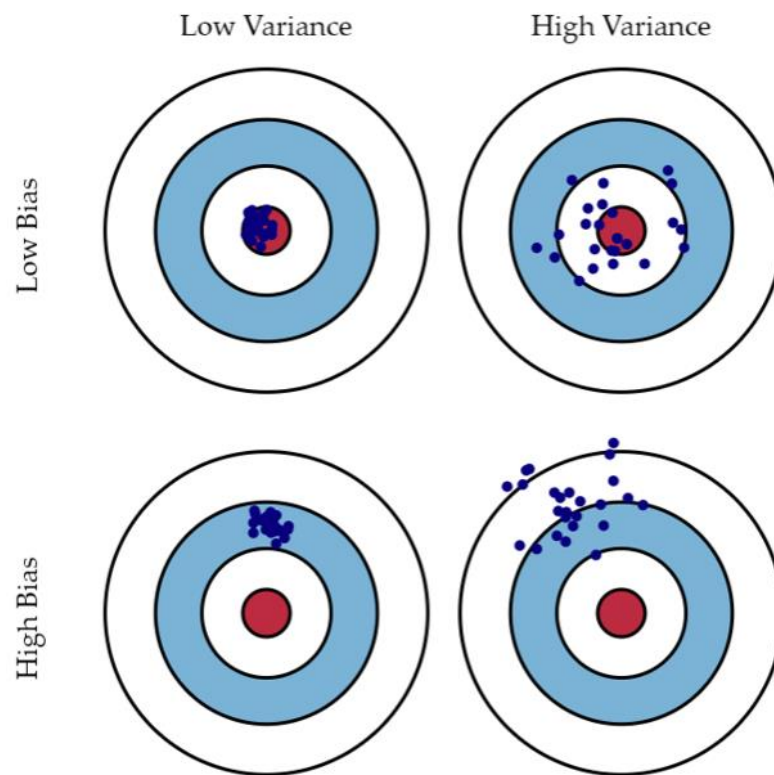
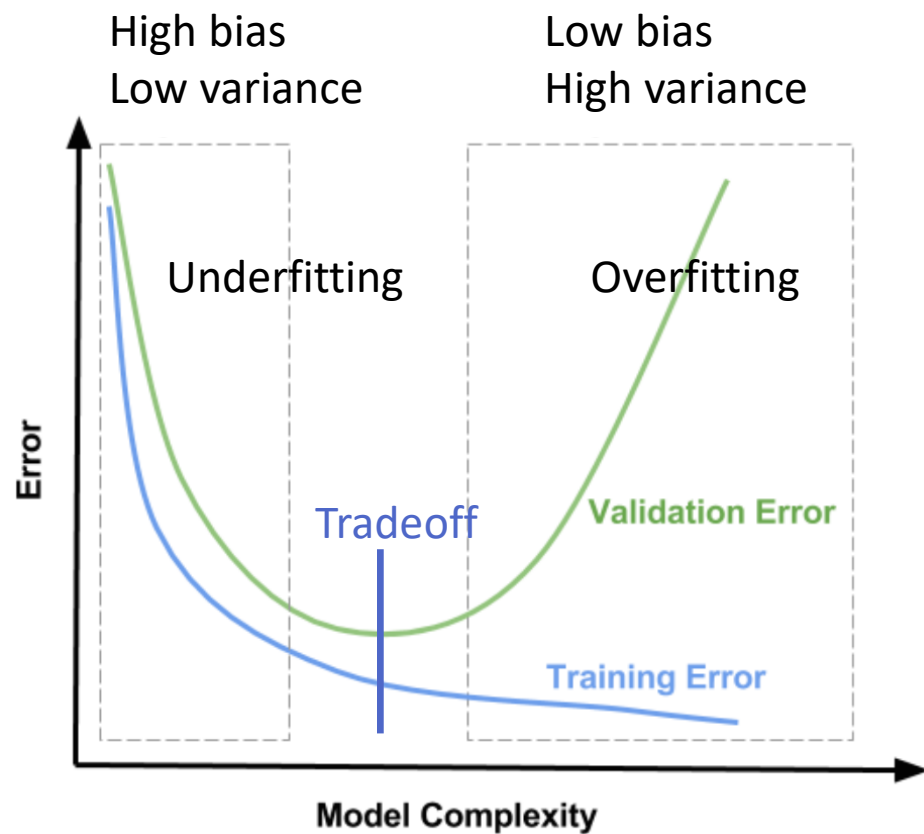
- ⊕ Bias = shooting angle
 - ▶ Distance from the actual target
 - ▶ Classifier underfits the data, i.e. fail to learn pattern
 - ▶ Classifier with many parameter setting tends to have more bias
 - ▶ Can be controlled by parameter adjustment

⊕ Variance = shooting force

- ▶ Variation of the hits
- ▶ Effect of training data
- ▶ Classifier that is sensitive to input data (i.e. overfitting) tends to have more variance
- ▶ Can be controlled by sampling (to cover all representatives) or use ensemble model

⊕ Noise = observed target location

- ▶ Effect of intrinsic noise in target classes
- ▶ For example, different classes share the same properties, so they are naturally difficult to distinguish

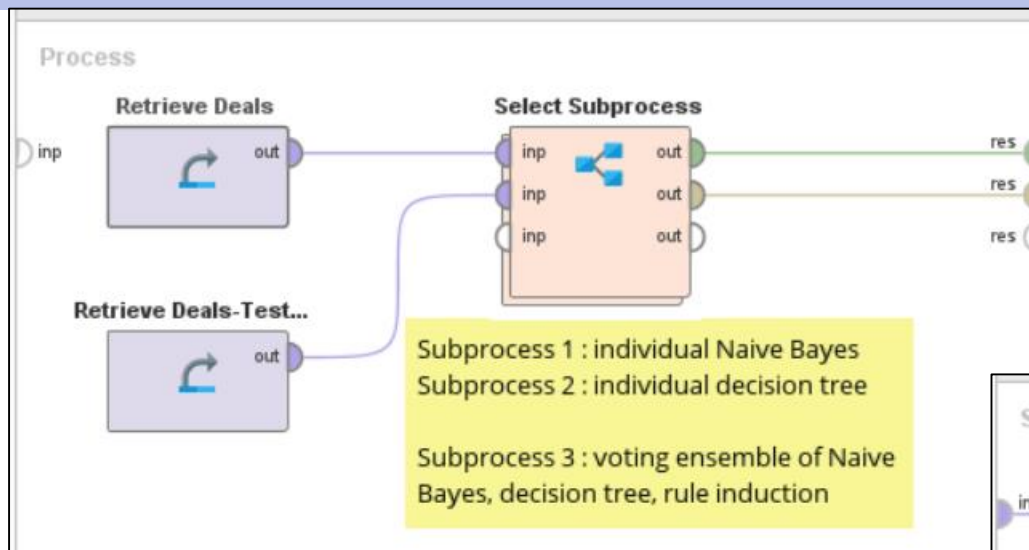


How to ensemble base classifiers ?

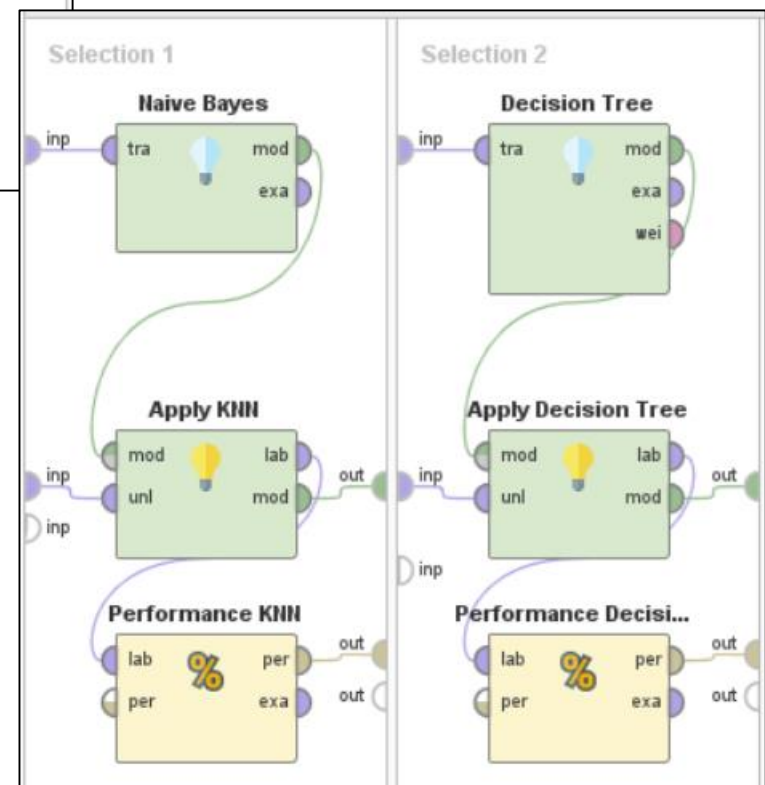
- ⊕ Use different types (or parameters) of base classifiers
 - ▶ Train base classifiers by the same training data
 - ▶ Ensemble methods → voting, stacking
- ⊕ Use different training records
 - ▶ Generate different training sets by sampling with replacement
 - ▶ Train base classifiers by different training data
 - ▶ Ensemble methods → bagging, boosting
- ⊕ Use different sets of attributes
 - ▶ Generate different training sets by sampling records with replacement and sampling attributes
 - ▶ Train base classifiers by different training data
 - ▶ Ensemble methods → random forest

- ⊕ Use different class labels
 - ▶ Convert multi-class problem to 2-class problem
 - ▶ Train multiple binary classifiers
 - ▶ Ensemble methods → 1 against 1, 1 against all
- ⊕ Ensemble methods involve training many base classifiers
 - ▶ Simple classifiers (e.g. decision stump, OneR) are commonly used
 - ▶ So that training overhead is not too high

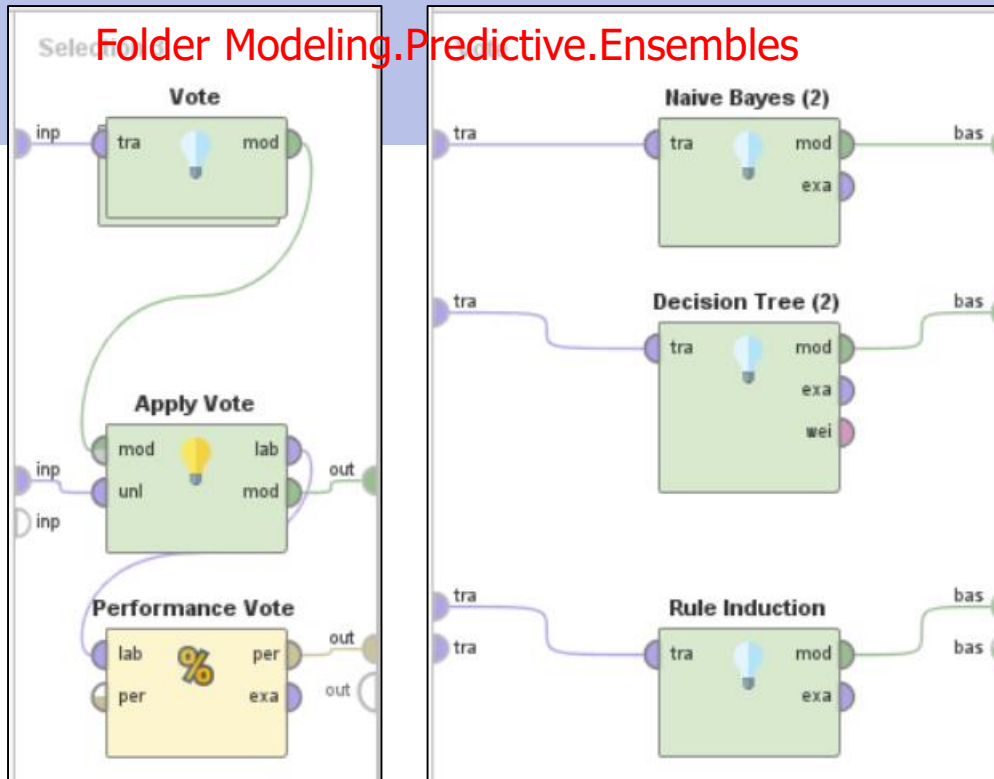
Example (1) : voting



Individual classifiers



Folder Modeling.Predictive.Ensembles



#base classifiers is odd to avoid tie because we have 2 classes

AttributeBasedVoting

Using the majority of the following attributes for prediction:

```
base_prediction0
base_prediction1
base_prediction2
```

The default value is no

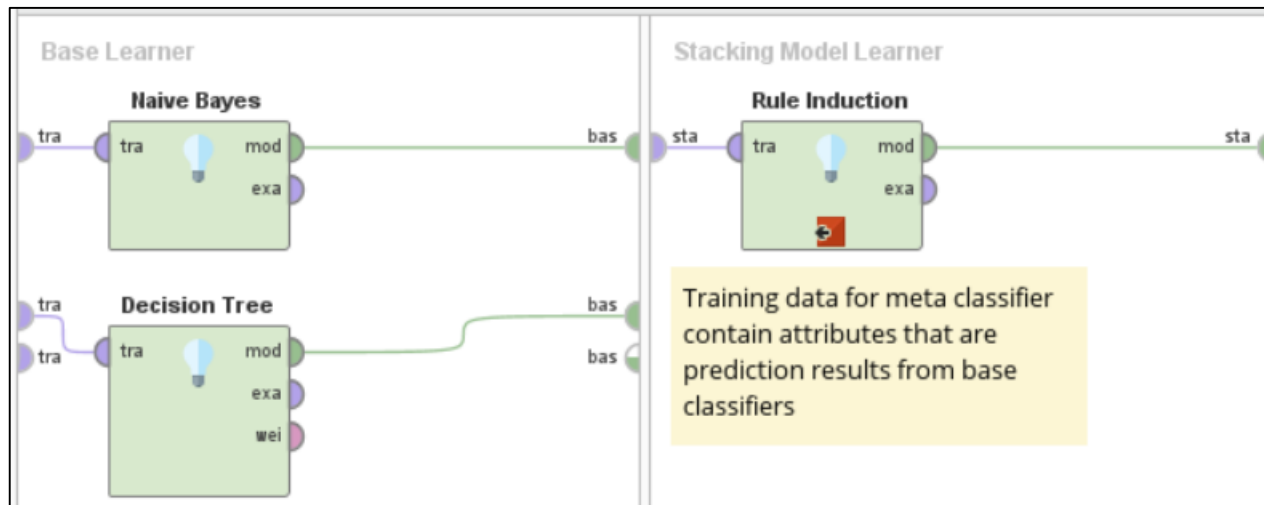
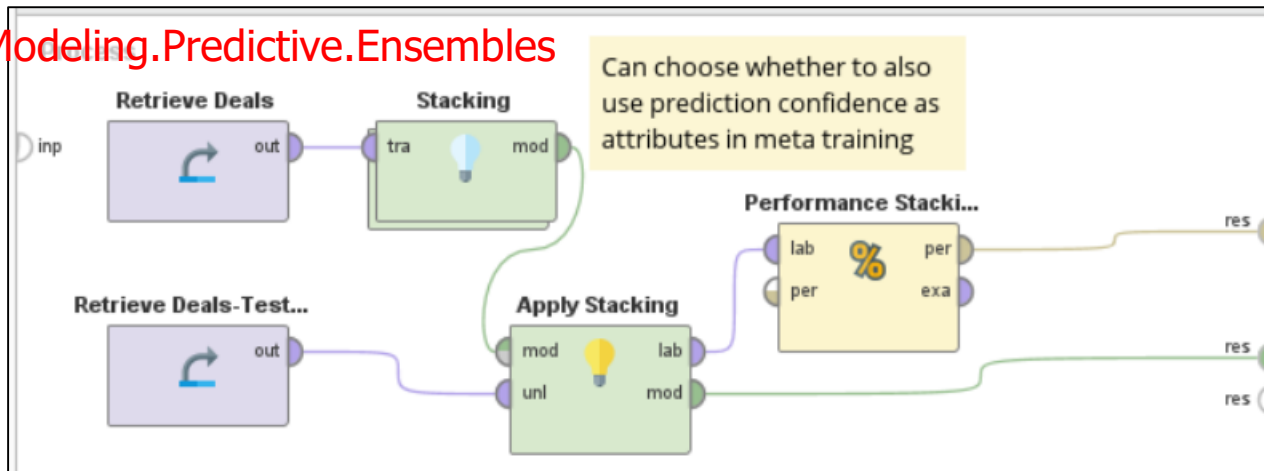
Final prediction is based on majority vote. In case of tie, default class = no (majority class in training data)

Stacking

- ⊕ Or stacked generalization
- ⊕ Split training data into 2 partitions
 - ▶ Base training
 - Use training partition (1) to train base classifiers
 - ▶ Meta training
 - Let each base classifier predict training partition (2)
 - Use prediction results by base classifiers to train meta classifier
 - Meta classifier learns prediction patterns of base classifiers that lead to the correct prediction
 - Simple classifiers are often used as meta classifiers

Example (2) : stacking

Folder Modeling.Predictive.Ensembles



Training partition (2) for training meta classifier

Row...	Future...	Age	Gender	Payment ...	base_con...	base_conf...	base_prediction0	base_conf...	base_conf...	base_prediction1
1	yes	64	male	credit card	0.432	0.568	no	0.847	0.153	yes
2	yes	35	male	cheque	0.529	0.471	yes	1	0	yes
3	yes	25	female	credit card	0.685	0.315	yes	0.995	0.005	yes
4	no	39	female	credit card	0.476	0.524	no	0	1	no
5	yes	39	male	credit card	0.882	0.118	yes	0.847	0.153	yes

Confidence of each class &
prediction by Naïve Bayes

Confidence of each class &
prediction by decision tree

RuleModel

```

if base_confidence_yes0 ≤ 0.229 then no (0 / 337)
if base_confidence_yes1 > 0.458 then yes (465 / 1)
else no (7 / 170)

correct: 972 out of 980 training examples.

```

Meta (stacking) model

Bagging (Bootstrap Aggregation)

- ⊕ Training n base classifiers of the same type

For each base classifier training {

1. Create training data or bootstrap by sampling with replacement
2. Use training data to train base classifier

}

- ⊕ Final prediction → by majority vote (or avg prediction confidence) from n base classifiers
- ⊕ Sampling with replacement
 - ▶ Some records are chosen many times while some are never chosen
 - ▶ On average, each bootstrap contains about 63% of original data (according to probability calculation)

⊕ For 1 sampling with replacement

- ▶ P(a record is selected once) $= 1/n$
- ▶ P(a record is not selected once) $= 1 - 1/n$

⊕ For n sampling with replacement

- ▶ P(a record is never selected) $= (1 - 1/n)^n$
- ▶ If n is very large : $\lim_{n \rightarrow \infty} (1 - 1/n)^n = 1/e \approx 0.368$
- ▶ P(a record is selected over n selections) $= 1 - 0.368 = 0.632$

Example : individual vs. bagging classifiers

⊕ Given original data

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	+	+	+	-	-	-	-	+	+	+

⊕ Suppose that we use decision tree & binary split only

⊕ Single classifier

- ▶ Classifier (1) : if ($x \leq 0.35$) then "+" else "-"

Accuracy on training data = 70%

- ▶ Classifier (2) : if ($x \leq 0.75$) then "-" else "+"

Accuracy on training data = 70%

⊕ Can ensemble classifier improve the accuracy rate ?

Base classifier (1) : if ($x \leq 0.35$) then "+" else "-"

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	+	+	+	+	-	-	-	-	+	+

Base classifier (2) : if ($x \leq 0.65$) then "+" else "+"

x	0.1	0.2	0.3	0.4	0.5	0.8	0.9	1	1	1
y	+	+	+	-	-	+	+	+	+	+

Base classifier (3) : if ($x \leq 0.30$) then "+" else "-"

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	+	+	+	-	-	-	-	-	+	+

Base classifier (4) : if ($x \leq 0.75$) then "-" else "+"

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	+	-	-	-	-	-	-	+	+	+

Base classifier (5) : if ($x \leq 0.75$) then "-" else "+"

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	+	+	-	-	-	-	-	+	+	+

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
---	-----	-----	-----	-----	-----	-----	-----	-----	-----	---

Prediction by base classifiers

Final decision = majority vote

y1	+	+	+	-	-	-	-	-	-	-
y2	+	+	+	+	+	+	+	+	+	+
y3	+	+	+	-	-	-	-	-	-	-
y4	-	-	-	-	-	-	-	+	+	+
y5	-	-	-	-	-	-	-	+	+	+
y	+	+	+	-	-	-	-	+	+	+

Actual

y	+	+	+	-	-	-	-	+	+	+
---	---	---	---	---	---	---	---	---	---	---

Accuracy = 100%

Boosting

- ⊕ Training n base classifiers of the same type

Let all training records have equal weights

For each base classifier training {

1. Create training data by weighted sampling with replacement
2. Use training data to train base classifier
3. Use base classifier to classify all training records
 - Calculate weight of this classifier (α). Low error = high α
 - Update weight of training records. Correctly predicted records get lower weights

}

- ⊕ Final prediction → weighted vote (by α) from n base classifiers

⊕ Weighted sampling with replacement

- ▶ Records with higher weights (the ones that get wrong prediction) will be sampled more often
- ▶ The next base classifiers focus more on hard-to-predict records
- ▶ But it doesn't mean later classifiers are better overall because they may be too focused on certain records. So, the prediction from each classifier is weighted by α

⊕ A popular implementation is AdaBoost (adaptive boosting)

Example : boosting classifiers

⊕ Given the same original data

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	+	+	+	-	-	-	-	+	+	+

⊕ Weight of each training record w

▶ First round $w = 1/n$; $n = \text{\#training records}$

▶ Next round $w_{\text{new}} = w_{\text{old}} \left(\frac{\varepsilon}{1-\varepsilon} \right)$; correctly predicted record

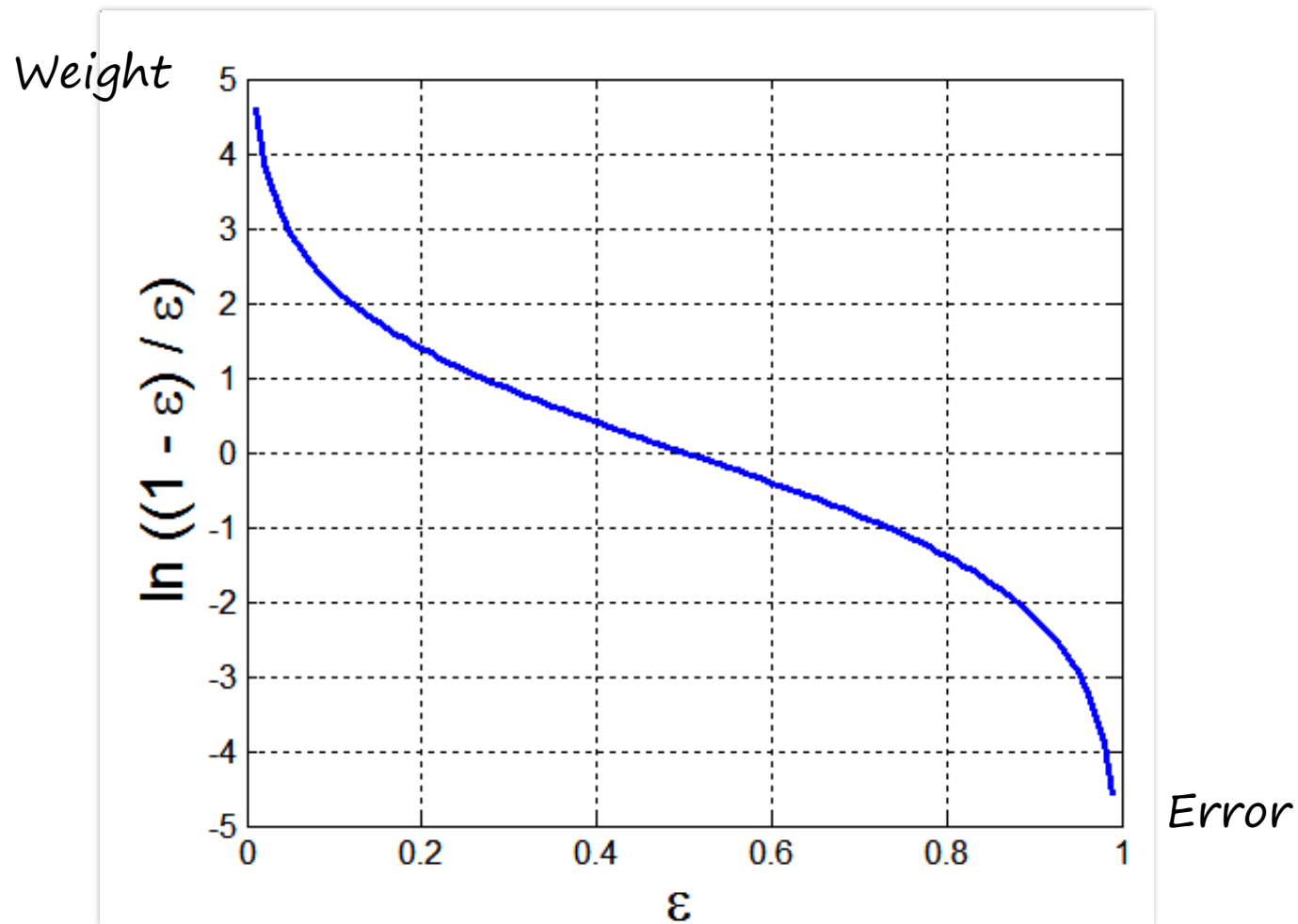
$w_{\text{new}} = w_{\text{old}}$; wrongly predicted record

▶ w_{new} is normalized by their sum, so that $\sum_{\text{all records}} (w_{\text{new}}) = 1$

⊕ Classifier error in each round $\varepsilon = \sum (w \text{ of wrongly predicted records})$

⊕ Weight of each base classifier $\alpha = \frac{1}{2} \ln \left(\frac{1-\varepsilon}{\varepsilon} \right)$

- ⊕ Higher error $\rightarrow$ lower weight or creditability for base classifier



All training records	x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
	y	+	+	+	-	-	-	-	+	+	+

Boosting round (1)

Initial weight	w1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
----------------	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

Base classifier (1) : if $(x \leq 0.35)$ then "+" else "-"

Randomed records	x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
	y1r	+	+	+	+	-	-	-	-	+	+
All records	x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
	y1a	+	+	+	-	-	-	-	-	-	-

Adjust weights for next round

$$\varepsilon = 0.1+0.1+0.1 = 0.3, \quad \alpha = 0.5 \ln (0.7/0.3) = 0.4236$$

New weight	w2	0.043	0.043	0.043	0.043	0.043	0.043	0.043	0.1	0.1	0.1
------------	----	-------	-------	-------	-------	-------	-------	-------	-----	-----	-----

Total w2 = 0.601

Normalized	w2	0.07	0.07	0.07	0.07	0.07	0.07	0.07	0.17	0.17	0.17
------------	----	------	------	------	------	------	------	------	------	------	------

Boosting round (2)

Weight	w2	0.07	0.07	0.07	0.07	0.07	0.07	0.07	0.17	0.17	0.17
--------	----	------	------	------	------	------	------	------	------	------	------

Base classifier (2) : if $(x \leq 0.75)$ then "-" else "+"

Randomed records	x	0.1	0.4	0.7	0.8	0.8	0.9	0.9	1	1	1
	y2r	-	-	-	+	+	+	+	+	+	+
All records	x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
	y2a	-	-	-	-	-	-	-	+	+	+

Adjust weights for next round

$$\varepsilon = 0.07+0.07+0.07 = 0.21, \quad \alpha = 0.5 \log (0.79/0.21) = 0.6625$$

New weight	w3	0.07	0.07	0.07	0.02	0.02	0.02	0.02	0.045	0.045	0.045
------------	----	------	------	------	------	------	------	------	-------	-------	-------

Total w3 = 0.425

Normalized	w3	0.165	0.165	0.165	0.047	0.047	0.047	0.047	0.106	0.106	0.106
------------	----	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Boosting round (3)

New	w3	0.165	0.165	0.165	0.047	0.047	0.047	0.047	0.106	0.106	0.106
-----	----	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Base classifier (3) : if $(x \leq 0.55)$ then "+" else "+"

Randomed instances	x	0.1	0.1	0.2	0.2	0.2	0.2	0.3	0.3	0.8	0.9
	y3r	+	+	+	+	+	+	+	+	+	+
All instances	x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
	y3a	+	+	+	+	+	+	+	+	+	+

Adjust weight for next round

$$\varepsilon = 0.047 + 0.047 + 0.047 + 0.047 = 0.188, \quad \alpha = 0.5 \log (0.812 / 0.188) = 0.7315$$

Suppose that we finish at round (3)

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
---	-----	-----	-----	-----	-----	-----	-----	-----	-----	---

Prediction by base classifier

Final decision = max { total weighted vote }

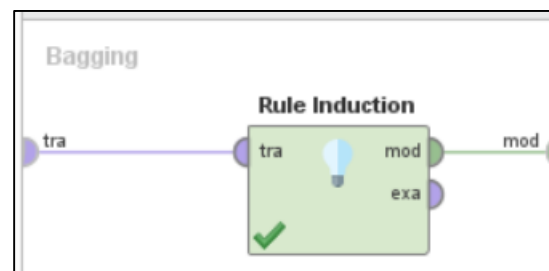
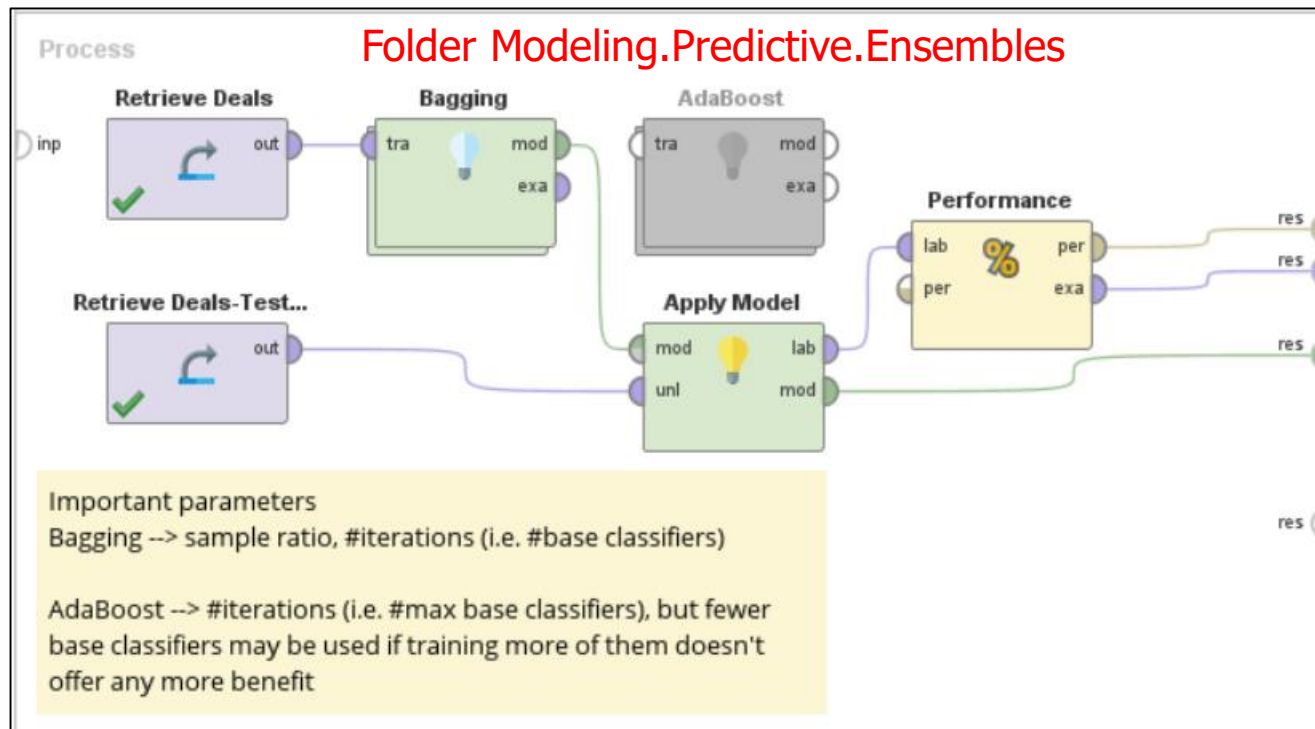
$y_1, \alpha_1 = 0.4236$	+	+	+	-	-	-	-	-	-	-
$y_2, \alpha_2 = 0.6625$	-	-	-	-	-	-	-	+	+	+
$y_3, \alpha_3 = 0.7315$	+	+	+	+	+	+	+	+	+	+
y	+	+	+	-	-	-	-	+	+	+

Actual

y	+	+	+	-	-	-	-	+	+	+
---	---	---	---	---	---	---	---	---	---	---

$$\begin{aligned}
 x = 0.1 & \Rightarrow y = 0.4236 - 0.6625 + 0.7315 & \Rightarrow \text{class } + \\
 x = 0.4 & \Rightarrow y = -0.4236 - 0.6625 + 0.7315 & \Rightarrow \text{class } - \\
 x = 0.8 & \Rightarrow y = -0.4236 + 0.6625 + 0.7315 & \Rightarrow \text{class } +
 \end{aligned}$$

Example (3) : bagging & boosting



Results from Bagging

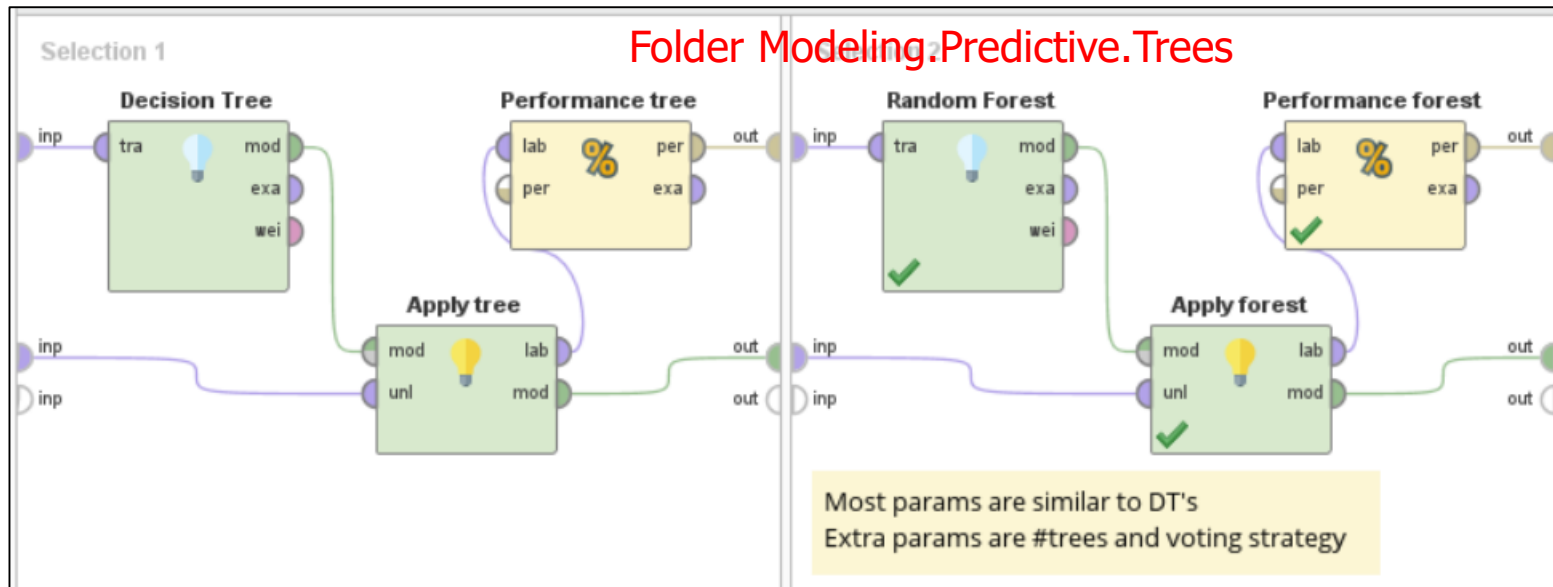
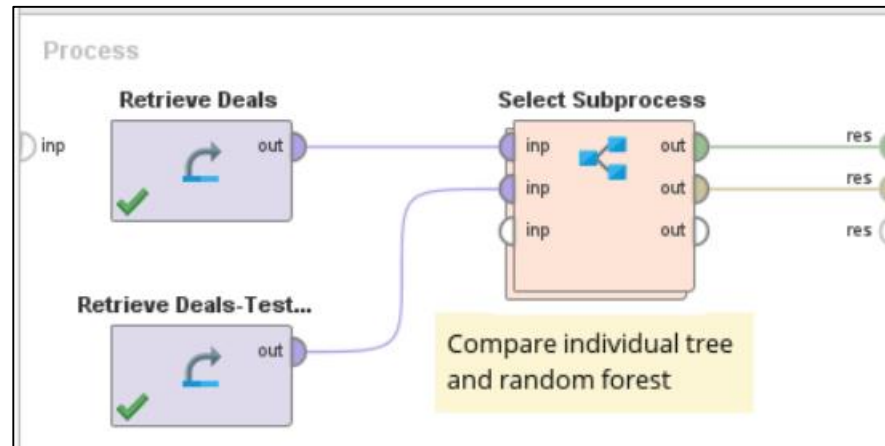
Row No.	Future Cust...	prediction(F...	confidence(yes)	confidence(no)	Age	Gender	Payment Me...
1	yes	yes	0.932	0.068	44	male	credit card
2	no	no	0	1.000	86	female	credit card
3	yes	yes	0.932	0.068	17	female	credit card
4	no	no	0	1.000	57	female	credit card
5	no	no	0	1.000	48	female	credit card

If averaging confidence parameter is set → prediction confidences will be averaged from all base classifiers

Random Forest

- ⌘ Bagging with decision trees as base classifiers. But each tree is built from different random subsets of attributes
- ⌘ Final prediction can be based on majority vote or weighted votes (by prediction confidence)
- ⌘ Popular ensemble method for large data sets with many attributes. It can be used for attribute selection (see Chapter 4)

Example (4) : random forest



Gradient Boosting Trees

- ⊕ Apply boosting concept to decision & regression trees
 - ▶ In each iteration, we train a new weak classifier to compensate the shortcomings of previous weak classifiers
 - ▶ AdaBoost → shortcomings are identified by weights (mispredicted records have higher weights & thus are sampled more often)
 - ▶ Gradient boosting → shortcomings are identified by gradient. Full data set is typically used to train base classifiers
- ⊕ After training each weak classifier
 - ▶ Calculate residuals : $e_i = y_i - \hat{y}_i$ and $E = \sum e_i$ for all records
 - ▶ Train the next weak classifier to learn the pattern of residuals
 - ▶ For classification task, $\hat{y}$ is prediction confidence

Weak learner 1 fits $f_1(X)$ to (X, Y) data

$$Y = f_1(X) + E_1$$

Weak learner 2 fits $f_2(X)$ to (X, E_1) data

$$E_1 = f_2(X) + E_2$$

Weak learner 3 fits $f_3(X)$ to (X, E_2) data

$$E_2 = f_3(X) + E_3$$

Strong learner

$$Y = f_1(X) + f_2(X) + f_3(X) + E_3$$

Adding classifier weight α_i

$$Y = \alpha_1 f_1(X) + \alpha_2 f_2(X) + \alpha_3 f_3(X) + E$$

- ⊕ $0 < \alpha_i \leq 1$ is learning rate i.e. how quickly error is corrected by a learner
- ⊕ Recall gradient descent in Chapter 6 → we try to reach optimal point by fitting the model along the gradient

By weight adjustment in neural network

*By function fitting in regression
(based on least squared method)*

*Residual is related
to negative gradient*

⊕ Residual vs. negative gradient

Let loss function based on squared error $L = \frac{1}{2} (Y - f(X))^2$

To minimize loss function by adjusting each $f(X)$

Partial derivative w.r.t. $f(X)$

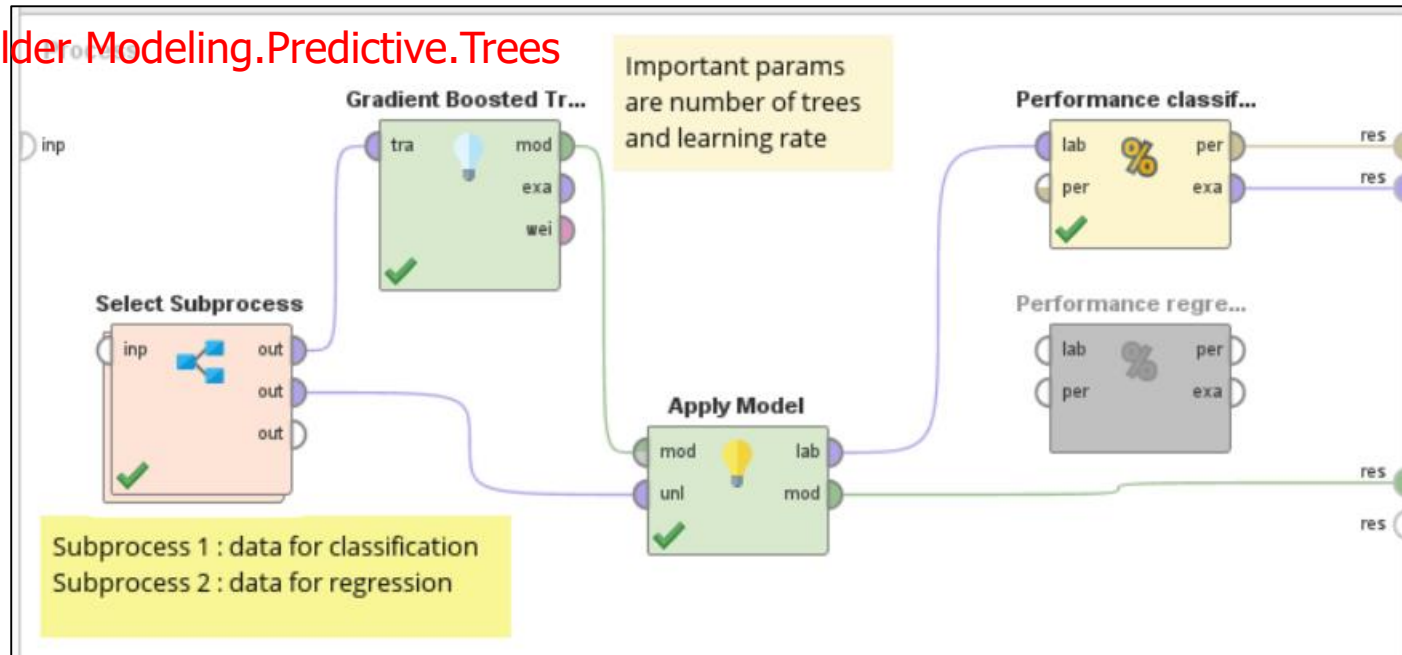
$$\frac{\partial L}{\partial f(X)} = (Y - f(X)) \frac{d(Y - f(X))}{d f(X)} = (Y - f(X))(-1)$$

$$-\frac{\partial L}{\partial f(X)} = Y - f(X)$$

But residual is not always equal to negative gradient (if we use different loss function). So in gradient boosting algorithm, we actually try to fit regression model to negative gradient

Example (5) : gradient boosting tree

Folder Modeling.Predictive.Trees



Multi-class vs. Binary Classification

⊕ Reasons to use binary classification

- ▶ Some methods support only binary classification e.g. SVM, logistic regression
- ▶ Class imbalance problem in multi-class classification → some classes are very small compared to the others
- ▶ Multi-class model is more complicated than binary model
- ▶ Classifying too many classes results in poor accuracy

⊕ From multi-class → binary classification

- ▶ One against one
- ▶ One against all

⊕ Suppose there are 4 classes { C1, C2, C3, C4 }

⊕ One against one

- ▶ Train binary classifier to distinguish between each pair of classes C1-C2, C1-C3, C1-C4, C2-C3, C2-C4, C3-C4
 - When training each binary classifier, irrelevant records from other classes are ignored
- ▶ Then, let all binary classifier predict a new record
 - E.g. predictions = (C1, C1, C1, C3, C2, C3)
 - Using majority votes, final prediction = C1
 - Or compare average prediction confidence of C1, C2, C3, C4 and choose the one with highest confidence

⊕ One against all

- ▶ Train binary classifier to predict whether records belong to a class or the others → C1-others, C2-others, C3-others, C4-others
- ▶ Then, let all binary classifier predict a new record
 - E.g. predictions = (C1, others, others, others)

	C1	C2	C3	C4
Classifier 1	1	0	0	0
Classifier 2	1	0	1	1
Classifier 3	1	1	0	1
Classifier 4	1	1	1	0

predict C1, only C1 gets 1

predict others, all except C2 gets 1

predict others, all except C3 gets 1

predict others, all except C4 gets 1

- Using majority votes, final prediction = C1
- Or choose class with highest prediction confidence

Comparison

- ⊕ In many practical cases, their performance are similar
- ⊕ One against one requires $\frac{n(n-1)}{2}$ classifiers for n classes
 - ▶ Training is more computationally expensive
 - ▶ More robust to class imbalance
- ⊕ One against all requires only n classifiers for n classes
 - ▶ Faster training, thus generally more popular
 - ▶ More sensitive to class imbalance problem → when classifying a very small class against all the others

Summary

- ⊕ **Voting and stacking** methods support different classifiers
 - ▶ Exploit their strengths & compensate for weaknesses
 - ▶ Voting gives equal weights to all base classifiers
 - ▶ Stacking learns prediction patterns of base classifiers
- ⊕ **Bagging and boosting** methods are useful when the classification is susceptible to data variation
 - ▶ Base classifiers are trained from different subsets of data (different focuses yield different biases) → data variation is neutralized by ensemble decision
 - ▶ Boosting can handle hard-to-predict data & thus correct mistake by previous base classifiers. But the model is more likely to be overfitting

⊕ Bagging and boosting (cont.)

- ▶ When there is little variation in data, these methods may not help
 - End up with all the same base classifiers
 - Sometimes results are worse → by sampling with replacement, base classifier is trained from 63% of original data, 37% is missing

⊕ Random forest = bagging of decision trees

- ▶ Handle both data variation and high dimensionality data
- ▶ Base classifiers are trained from different subsets of attributes → important attributes can be determined from ensemble decision

⊕ Gradient boosting tree = boosting of decision/regression trees

- ▶ All base classifiers are trained from full data set. But they learn to predict residuals left by previous base classifiers